

C# & OOPS (Day -4)



Authorized & published by
Itlize Global LLC

Table Of Contents

- Working Project Examples of (C# and OOPs):
 - Abstraction, Polymorphism and inheritance
 - Abstract Class
 - Interfaces
 - Static Classes
 - Constructor
 - Collection
 - Generics

C# and Object Oriented Programming

History of C#

- Originated by Microsoft as a response to Java.
 - Initial public release in 2000
- Language name inspired by musical note C#.
 - A “step above” C/C++ (and Java)
- Lead designers: Anders Hejlsberg, Scott Wiltamuth
- C sharp standardized via ECMA and ISO
 - While Microsoft retains architectural control

C# vs NET

- C# is a programming language
- .NET is a framework for building applications on Windows

CLASS

Data

Methods

CAR

Make

Model

Color

StartEngine()

Move()

Features of C#

- Unified object system
 - Everything is an object including primitives.
- Single inheritance
- Interfaces
 - Specify methods & interfaces, but not implementation.
- Structs
 - A restricted, lightweight (efficient) type
- Delegates
 - Expressive typesafe function pointer.
 - Useful for strategy and observer design patterns
- Preprocessor directives

First C# Program

- **using**: bring in namespaces, like import in Java.
- **namespace**: disambiguation of names, like C++ naming.
- **class**: single inheritance up to object, same as C++ or Java..
- It contains a single method, called Main.
- A **method** performs WriteLine action.
- The WriteLine method belongs to the System.Console object.
- The keyword “static” means that the method Main can be called even if there is no current instance of the class. It’s a class method, not an instance method.

```
using System;
namespace Test
{
    class ExampleClass
    {
        static void Main()
        {
            System.Console.WriteLine("Hello, world!");
        }
    }
}
```

Syntax

- Case-sensitive
- Ignoring whitespaces
 - including sequences of space, tab, linefeed, carriage return.
- Statements are terminated with **semicolons** (;).
- Code blocks are enclosed by **curly braces** ({}).
- types of comments:
 - `/* lines of comments */`
 - `// single line comment`
 - `/// <comment_in_xml>`
 - Automatic XML commenting facility

Variables

- Syntax: **type variable-name [= initialization-expression];**
- Examples:
 - `int number = 0;`
 - `string name;`
 - `float myfloat = 0.5f;`
 - `bool isValid = true;`
 - `const int freezingPoint = 32;`
- Variables must be initialized or assigned to before first use
- Class members take a visibility operator beforehand (private by default)
- Constants cannot be changed

Enumerations

- Syntax: **enum identifier [: base-type] { enumerator-list }**
- Example:
 - enum Week
 {
 Monday = 1, Tuesday = 2, Wednesday = 3, ...
 }
- Base type can be any integral type (ushort, long) except for char
- Defaults to int
- Must cast to int to display in WriteLine
 - Example: (int)Week.Monday

Enum Example

```
namespace System.Net
{
    public enum HttpStatusCode
    {
        Continue = 100,
        SwitchingProtocols = 101,
        OK = 200,
        Created = 201,
        Accepted = 202,
        NonAuthoritativeInformation = 203,
        NoContent = 204,
        ResetContent = 205,
        PartialContent = 206,
        MultipleChoices = 300,
        Ambiguous = 300,
        MovedPermanently = 301,
        Moved = 301,
        Found = 302,
        Redirect = 302,
        SeeOther = 303,
        RedirectMethod = 303,
        NotModified = 304,
        UseProxy = 305,
        Unused = 306,
        TemporaryRedirect = 307,
        RedirectKeepVerb = 307,
```

```
        RedirectKeepVerb = 307,
        BadRequest = 400,
        Unauthorized = 401,
        PaymentRequired = 402,
        Forbidden = 403,
        NotFound = 404,
        MethodNotAllowed = 405,
        NotAcceptable = 406,
        ProxyAuthenticationRequired = 407,
        RequestTimeout = 408,
        Conflict = 409,
        Gone = 410,
        LengthRequired = 411,
        PreconditionFailed = 412,
        RequestEntityTooLarge = 413,
        RequestUriTooLong = 414,
        UnsupportedMediaType = 415,
        RequestedRangeNotSatisfiable = 416,
        ExpectationFailed = 417,
        UpgradeRequired = 426,
        InternalServerError = 500,
        NotImplemented = 501,
        BadGateway = 502,
        ServiceUnavailable = 503,
        GatewayTimeout = 504,
        HttpVersionNotSupported = 505
    }
}
```

Conditionals

- Syntax: **if (expression) statement1 [else statement2]**
- Example:
 - `if (i < 0) { System.Console.WriteLine("i is negative"); }`
`else {System.Console.WriteLine("i is positive or zero");}`
- C# supports C/C++/Java syntax for “if ” statement
- Expression must evaluate to a bool value
 - No integer expressions here
- `==` means “equal to” for boolean comparison
 - `if (i == 0) // if i equals to 0`
 - `if (i = 0) // error, since i = 0 is not a boolean expression`

Switch statement

- Syntax: **switch (expression) { case constant-expression: statement(s);
Jump-statement [default: statement(s);]}**

- Alternative to if
- Typically use break
- Can use goto to continue to another case

Example:

```
const int raining = 1;
const int snowing = 0;
int weather = snowing;
switch (weather) {
    case snowing:
        System.Console.WriteLine("It is
        snowing!");
        goto case raining;
    case raining:
        System.Console.WriteLine("I am
        wet!");
        break;
    default:
        System.Console.WriteLine
        ("Weather OK");
        break;
}
```

Conditionals (other types)

Statement	Description
if statement ↗	An if statement consists of a boolean expression followed by one or more statements.
if...else statement ↗	An if statement can be followed by an optional else statement , which executes when the boolean expression is false.
nested if statements ↗	You can use one if or else if statement inside another if or else if statement(s).
switch statement ↗	A switch statement allows a variable to be tested for equality against a list of values.
nested switch statements ↗	You can use one switch statement inside another switch statement(s).

The ?: Operator

- Syntax: **Exp1 ? Exp2 : Exp3;**
- Exp1, Exp2, and Exp3 are expressions.
- The value of a ? expression is determined as follows: Exp1 is evaluated. If it is true, then Exp2 is evaluated and becomes the value of the entire ? expression. If Exp1 is false, then Exp3 is evaluated and its value becomes the value of the expression.

Loops

Loop Type	Description
while loop 	It repeats a statement or a group of statements while a given condition is true. It tests the condition before executing the loop body.
for loop 	It executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
do...while loop 	It is similar to a while statement, except that it tests the condition at the end of the loop body
nested loops 	You can use one or more loop inside any another while, for or do..while loop.

Loop Control Statements

- Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed.

Control Statement	Description
break statement 	Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
continue statement 	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

Infinite Loop

- A loop becomes infinite loop if a condition never becomes false. The for loop is traditionally used for this purpose. Since none of the three expressions that form the for loop are required, you can make an endless loop by leaving the conditional expression empty.
- Example:

```
for ( ; ; )  
{  
    Console.WriteLine("Hey! I am Trapped");  
}
```

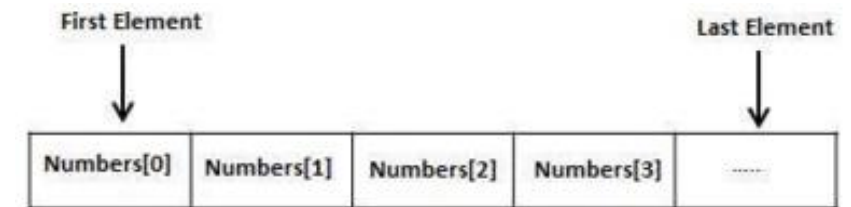
Nullables

- C# provides a special data type, the nullable type, to which you can assign normal range of values as well as null values.
- Syntax: **< data_type> ? <variable_name> = null;**
- Example:
 - `int? num1 = null;`
 - `double? num2 = 3.14;`
- The Null Coalescing Operator (??)
 - Example:

```
using System;
namespace CalculatorApplication
{
    class NullablesAtShow
    {
        static void Main(string[] args)
        {
            double? num1 = null;
            double? num2 = 3.14157;
            double num3;
            num3 = num1 ?? 5.34;
            Console.WriteLine(" Value of num3: {0}", num3);
            num3 = num2 ?? 5.34;
            Console.WriteLine(" Value of num3: {0}", num3);
            Console.ReadLine();
        }
    }
}
```

Arrays

- Array stores a fixed-size sequential collection of elements.
- Elements are the same type.
- Consist of contiguous memory locations.
- Syntax: **datatype[] arrayName;**
- Examples:
 - `double[] balance = new double[10];`
 - `double[] balance = { 23.0, 45.2, 34.0 };`
- Accessing Array Elements:
 - `double salary = balance[9];`
- Using the foreach Loop:



```
int [] n = new int[10]; /* n is an array of 10 integers */  
  
/* initialize elements of array n */  
for ( int i = 0; i < 10; i++ )  
{  
    n[i] = i + 100;  
}  
  
/* output each array element's value */  
foreach (int j in n )  
{  
    int i = j-100;  
    Console.WriteLine("Element[{0}] = {1}", i, j);  
    i++;  
}  
Console.ReadKey();
```

Strings

- Strings is an array of characters.
- Creating a String Object.
- Common Methods of the String Class.
 - String.Compare(strA, strB)
 - String.Concat(strA, strB)
 - strA.Contains(strB)
 - strA.Substring(0,3)
 - String.Join("\n", string_array)

```
//from string literal and string concatenation
string fname, lname;
fname = "Rowan";
lname = "Atkinson";

string fullname = fname + lname;
Console.WriteLine("Full Name: {0}", fullname);

//by using string constructor
char[] letters = { 'H', 'e', 'l', 'l', 'o' };
string greetings = new string(letters);
Console.WriteLine("Greetings: {0}", greetings);

//methods returning string
string[] sarray = { "Hello", "From", "Tutorials", "Point" };
string message = String.Join(" ", sarray);
Console.WriteLine("Message: {0}", message);

//formatting method to convert a value
DateTime waiting = new DateTime(2012, 10, 10, 17, 58, 1);
string chat = String.Format("Message sent at {0:t} on {0:D}", waiting);
Console.WriteLine("Message: {0}", chat);
```

Structures

- A value type data type.
- A single variable hold related data of various data types.
- Defining a Structure.
- The use of the structure.
- Features of C# Structures:
 - can have methods, fields, indexers, properties, operator methods, and events.
 - can have defined constructors, but not destructors, default constructor is automatically defined and cannot be changed.
 - cannot inherit other structures or classes.
 - cannot be used as a base for other structures or classes.
 - can implement one or more interfaces.
 - cannot be specified as abstract, virtual, or protected.
 - can be instantiated without using the **new** operator.
 - if the new operator is not used, the fields remain unassigned and the object cannot be used until all the fields are initialized.

```
struct Books
{
    public string title;
    public string author;
    public string subject;
    public int book_id;
};
```

```
Books Book1;    /* Declare Book1 of type Book */
Books Book2;    /* Declare Book2 of type Book */

/* book 1 specification */
Book1.title = "C Programming";
Book1.author = "Nuha Ali";
Book1.subject = "C Programming Tutorial";
Book1.book_id = 6495407;

/* book 2 specification */
Book2.title = "Telecom Billing";
Book2.author = "Zara Ali";
Book2.subject = "Telecom Billing Tutorial";
Book2.book_id = 6495700;
```

Exception Handling

An exception is a problem that arises during the execution of a program. A C# exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

Exceptions provide a way to transfer control from one part of a program to another. C# exception handling is built upon four keywords: **try**, **catch**, **finally**, and **throw**.

- **try:** A try block identifies a block of code for which particular exceptions is activated. It is followed by one or more catch blocks.

- **catch:** A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The catch keyword indicates the catching of an exception.
- **finally:** Used to execute a given set of statements, whether an exception is thrown or not thrown.
- **throw:** A program throws an exception when a problem shows up. This is done using a throw keyword.

```
using System;

namespace ErrorHandlingApplication {

    class DivNumbers {
        int result;

        DivNumbers() {
            result = 0;
        }

        public void division(int num1, int num2) {
            try {
                result = num1 / num2;
            } catch (DivideByZeroException e) {
                Console.WriteLine("Exception caught: {0}", e);
            } finally {
                Console.WriteLine("Result: {0}", result);
            }
        }

        static void Main(string[] args) {
            DivNumbers d = new DivNumbers();
            d.division(25, 0);
            Console.ReadKey();
        }
    }
}
```

C# and Object Oriented Programming

Object Oriented Application: It's a collection of related objects, communicating with each other, exchanging messages with each other in a controlled environment as per the rules of the business.

Every object oriented language should have three features:

- Encapsulation
- Inheritance
- Polymorphism

Classes and Objects

- A class combines together:
 - Data: class variables
 - Behavior: methods
- A key feature of object-oriented languages.
 - Procedural languages, such as C, did not require clustering of data and behavior.
- Class/instance distinction
 - Class defines variables & methods.
 - Instantiation of class (creating objects) is required to use variables and methods.
 - Analogy: a jelly bean mold (class) can be used to create a large number of jelly beans (objects, instances of the class)

Namespaces

- A **namespace** is designed for providing a way to keep one set of names separate from another. The class names declared in one namespace does not conflict with the same class names declared in another.

```
using System;
namespace first_space
{
    class namespace_cl
    {
        public void func()
        {
            Console.WriteLine("Inside first_space");
        }
    }
}

namespace second_space
{
    class namespace_cl
    {
        public void func()
        {
            Console.WriteLine("Inside second_space");
        }
    }
}

class TestClass
{
    static void Main(string[] args)
    {
        first_space.namespace_cl fc = new first_space.namespace_cl();
        second_space.namespace_cl sc = new second_space.namespace_cl();
        fc.func();
        sc.func();
        Console.ReadKey();
    }
}
```

Defining a class

- Syntax : **[attributes] [access-modifiers] class identifier [:base-class [,interface(s)]]{ class-body }**
- Simple example:

```
class A {  
    int num = 0; // a simple variable  
    A(int initial_num) { num = initial_num; } // set initial value of num  
}
```

- Attributes: used to add metadata to a class : can safely be ignored.
- Access modifiers: can be public, private, protected, internal, protected internal.
- Base-class: indicates (optional) parent for inheritance.
- Interfaces: Indicates (optional) interfaces that supply method signatures that need to be implemented in the class.
- Class-body: code for the variables and methods of the class.

Encapsulation

- The process of enclosing one or more items within a physical or logical package.
- Prevents access to implementation details.
- Implemented by using access specifiers.
 - Defines the scope and visibility of a class member.
- Types of access specifiers:
 - **Public:** can be accessed outside the class.
 - **Private:** can only be accessed within current class.
 - **Protected:** can be accessed within current class and child class.
 - **Internal:** can be accessed in the current assembly.
 - **Protected Internal:** can be accessed by child class within the same application.

Inheritance

- The Process of acquiring the existing functionality of the parent and with new added features and functionality by a child object is called inheritance.
- The advantages of inheritance are Generalization, Extensibility and Reusability.
- For example: A calculator is a generalized form of mathematical operations where as a Scientific calculator is an Extended and Specific form.

Inheritance Example

```
class A {  
    public void display_one() {  
        System.Console.WriteLine("I come from A");  
    }  
}  
class B : A {  
    public void display_two() {  
        System.Console.WriteLine("I come from B, child of A");  
    }  
}  
class App {  
    static void Main() {  
        A a = new A(); // Create instance of A  
        B b = new B(); // Create instance of B  
  
        a.display_one(); // I come from A  
        b.display_one(); // I come from A  
        b.display_two(); // I come from B, child of A  
    }  
}
```

Polymorphism

- "Poly" means "many" and "morph" means "form".
- An object in different forms and in each form it exhibits the same functionality but implemented in different ways.
- A parent class variable taking the form of child class objects and using this variable invoking the functionality of object this variable has actually taken the form (either of the child class object)

Polymorphism

- Referred as 'one interface, multiple functions'.
- Can be static or dynamic:
 - **Static** polymorphism: the response to a function is determined at the compile time.
 - **Dynamic** polymorphism: decided at run-time.

Static Polymorphism

- Early binding (static binding): the mechanism of linking a function with an object during compile time. Including:
 - Function overloading
 - Operator overloading

```
class Printdata
{
    void print(int i)
    {
        Console.WriteLine("Printing int: {0}", i );
    }

    void print(double f)
    {
        Console.WriteLine("Printing float: {0}" , f);
    }

    void print(string s)
    {
        Console.WriteLine("Printing string: {0}", s);
    }
}
```

```
static void Main(string[] args)
{
    Printdata p = new Printdata();

    // Call print to print integer
    p.print(5);

    // Call print to print float
    p.print(500.263);

    // Call print to print string
    p.print("Hello C++");
    Console.ReadKey();
}
```

Dynamic Polymorphism

- Creating classes that are used to provide partial class implementation of an interface.
- Implementation is completed when a derived class inherits from it.
- Abstract classes contain abstract methods, which are implemented by the derived class.
- Rules of abstract classes:
 - You cannot create an instance of an abstract class
 - You cannot declare an abstract method outside an abstract class
 - When a class is declared sealed, it cannot be inherited, abstract classes cannot be declared sealed.

Dynamic Polymorphism (Examples)

Abstract class example:

```
abstract class Shape
{
    public abstract int area();
}
class Rectangle: Shape
{
    private int length;
    private int width;
    public Rectangle( int a=0, int b=0)
    {
        length = a;
        width = b;
    }
    public override int area ()
    {
        Console.WriteLine("Rectangle class area :");
        return (width * length);
    }
}
```

Abstract class with **virtual function** example:

- Virtual functions are used when you have a function defined you want to be implemented in an inherited class.
- Virtual functions could be implemented differently in different inherited class and the call to these functions will be decided at **runtime**.

```
class Shape
{
    protected int width, height;
    public Shape( int a=0, int b=0)
    {
        width = a;
        height = b;
    }
    public virtual int area()
    {
        Console.WriteLine("Parent class area :");
        return 0;
    }
}
class Rectangle: Shape
{
    public Rectangle( int a=0, int b=0): base(a, b)
    {
    }
    public override int area ()
    {
        Console.WriteLine("Rectangle class area :");
        return (width * height);
    }
}
```

ABSTRACTION

```
abstract class Shape
{
    public abstract int GetArea();
}

class Square : Shape
{
    int side;

    public Square(int n) => side = n;

    // GetArea method is required to avoid a compile-time error.
    public override int GetArea() => side * side;

    static void Main()
    {
        var sq = new Square(12);
        Console.WriteLine($"Area of the square = {sq.GetArea()}");
    }
}

// Output: Area of the square = 144
```

- Generally, we use abstract class at the time of *inheritance*.
- A user must use the *override* keyword before the method which is declared as abstract in child class, the abstract class is used to inherit in the child class.
- An abstract class cannot be inherited by structures.
- It can contain constructors or destructors.
- It can implement functions with non-Abstract methods.
- It cannot support multiple inheritance.
- It can't be static.

Interfaces

An interface is defined as a syntactical contract that all the classes inheriting the interface should follow. The interface defines the 'what' part of the syntactical contract and the deriving classes define the 'how' part of the syntactical contract.

Interfaces define properties, methods, and events, which are the members of the interface. Interfaces contain only the declaration of the members. It is the responsibility of the deriving class to define the members. It often helps in providing a standard structure that the deriving classes would follow.

```
public interface ITransactions
{
    // interface members
    void showTransaction();
    double getAmount();
}

public class Transaction : ITransactions
{
    private string tCode;
    private string date;
    private double amount;
    public Transaction()
    {
        tCode = " ";
        date = " ";
        amount = 0.0;
    }

    public Transaction(string c, string d, double a)
    {
        tCode = c;
        date = d;
        amount = a;
    }

    public double getAmount()
    {
        return amount;
    }

    public void showTransaction()
    {
        Console.WriteLine("Transaction: {0}", tCode);
        Console.WriteLine("Date: {0}", date);
        Console.WriteLine("Amount: {0}", getAmount());
    }
}
```

Static Member of Class

We can define class members as static using the static keyword. When we declare a member of a class as static, it means no matter how many objects of the class are created, there is only one copy of the static member.

- Static members can be variables and functions.

```
using System;
namespace StaticVarApplication
{
    class StaticVar
    {
        public static int num;
        public void count()
        {
            num++;
        }
        public int getNum()
        {
            return num;
        }
    }
}
```

```
class StaticTester
{
    static void Main(string[] args)
    {
        StaticVar s1 = new StaticVar();
        StaticVar s2 = new StaticVar();
        s1.count();
        s1.count();
        s1.count();
        s2.count();
        s2.count();
        s2.count();
        Console.WriteLine("Variable num for s1: {0}", s1.getNum());
        Console.WriteLine("Variable num for s2: {0}", s2.getNum());
        Console.ReadKey();
    }
}
```

Results:

```
Variable num for s1: 6
Variable num for s2: 6
```

Constructors versus Destructors

Constructors

Executed whenever a new object of that class is created.

Has exactly the same name as class name.

has no return type.

Can have parameters.

Can be overloaded.

```
class Line
{
    private double length; // Length of a line
    public Line()
    {
        Console.WriteLine("Object is being created");
    }
}

class Line
{
    private double length; // Length of a line
    public Line(double len) //Parameterized constructor
    {
        Console.WriteLine("Object is being created, length = {0}", len);
        length = len;
    }
}
```

Destructors

Executed whenever an object of that class goes out of scope.

Has exactly the same name as class name with a prefixed tilde (~).

Memory was released before exiting the program.

Cannot be inherited or overloaded.

```
class Line
{
    private double length; // Length of a line
    public Line() // constructor
    {
        Console.WriteLine("Object is being created");
    }
    ~Line() //destructor
    {
        Console.WriteLine("Object is being deleted");
    }
}
```

Collections

- Collection classes are specialized classes for data storage and retrieval. These classes provide support for stacks, queues, lists, and hash tables. Most collection classes implement the same interfaces.
- Collection classes serve various purposes, such as allocating memory dynamically to elements and accessing a list of items on the basis of an index etc. These classes create collections of objects of the Object class, which is the base class for all data types in C#.

Various Collection Classes:

- **ArrayList:** Represents ordered collection of an object, an alternative to an array. can add and remove items from a list at a specified position using an index.
- **Hashtable:** Uses a key to access the elements in the collection. Each item in the hashtable is a key/value pair.
- **SortedList:** Uses a key as well as an index to access the items in a list. A combination of an array and a hashtable.
- **Stack:** last in, first out collection of object.
- **Queue:** first in, first out collection of object.
- **BitArray:** an array of the binary representation using the values 1 and 0.

Generics

- Allow you to delay the specification of the data type of programming elements in a class or a method, until it is actually used in the program.

Example of generic array:

```
public class MyGenericArray<T>
{
    private T[] array;
    public MyGenericArray(int size)
    {
        array = new T[size + 1];
    }

    public T getItem(int index)
    {
        return array[index];
    }

    public void setItem(int index, T value)
    {
        array[index] = value;
    }
}
```

Features of Generics

Generics is a technique that enriches your programs in the following ways:

- It helps you to maximize code reuse, type safety, and performance.
- You can create generic collection classes. The .NET Framework class library contains several new generic collection classes in the System.Collections.Generic namespace. You may use these generic collection classes instead of the collection classes in the System.Collections namespace.
- You can create your own generic interfaces, classes, methods, events, and delegates.
- You may create generic classes constrained to enable access to methods on particular data types.
- You may get information on the types used in a generic data type at runtime by means of reflection.

Generic Methods (examples)

```
static void Swap<T>(ref T lhs, ref T rhs)
{
    T temp;
    temp = lhs;
    lhs = rhs;
    rhs = temp;
}

static void Main(string[] args)
{
    int a, b;
    char c, d;
    a = 10;
    b = 20;
    c = 'I';
    d = 'V';

    //display values before swap:
    Console.WriteLine("Int values before calling swap:");
    Console.WriteLine("a = {0}, b = {1}", a, b);
    Console.WriteLine("Char values before calling swap:");
    Console.WriteLine("c = {0}, d = {1}", c, d);

    //call swap
    Swap<int>(ref a, ref b);
    Swap<char>(ref c, ref d);

    //display values after swap:
    Console.WriteLine("Int values after calling swap:");
    Console.WriteLine("a = {0}, b = {1}", a, b);
    Console.WriteLine("Char values after calling swap:");
    Console.WriteLine("c = {0}, d = {1}", c, d);

    Console.ReadKey();
}
```

Output:

```
Int values before calling swap:
a = 10, b = 20
Char values before calling swap:
c = I, d = V
Int values after calling swap:
a = 20, b = 10
Char values after calling swap:
c = V, d = I
```

Events

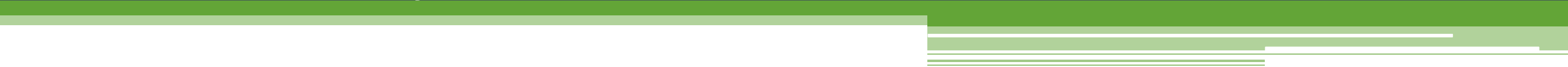
- Events are user actions such as key press, clicks, mouse movements, etc.
- Declared and raised in a class and associated with the event handlers using delegates within the same class or some other class.
- Events use the publisher-subscriber model.
- A **publisher** is an object that contains the definition of the event and the delegate. The event-delegate association is also defined in this object. A publisher class object invokes the event and it is notified to other objects.
- A **subscriber** is an object that accepts the event and provides an event handler. The delegate in the publisher class invokes the method (event handler) of the subscriber class.
- To declare an event inside a class, first a delegate type for the event must be declared. For example,

```
public delegate void BoilerLogHandler(string status);
```

Wrap-up

- Summarize important points.
- Questions.

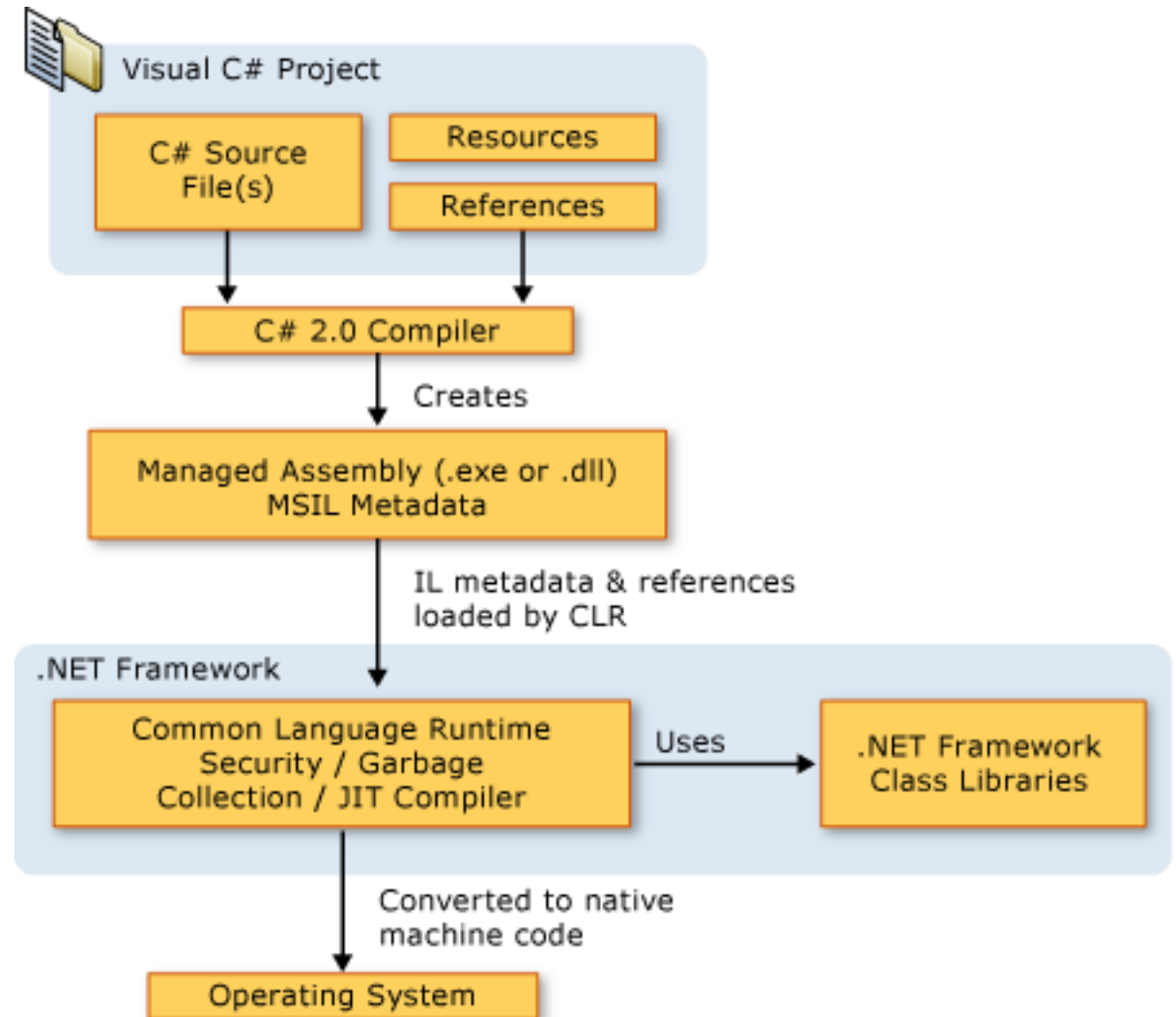
Further Study



Authorized & published by
Itlize Global LLC

CLR and JIT compiling

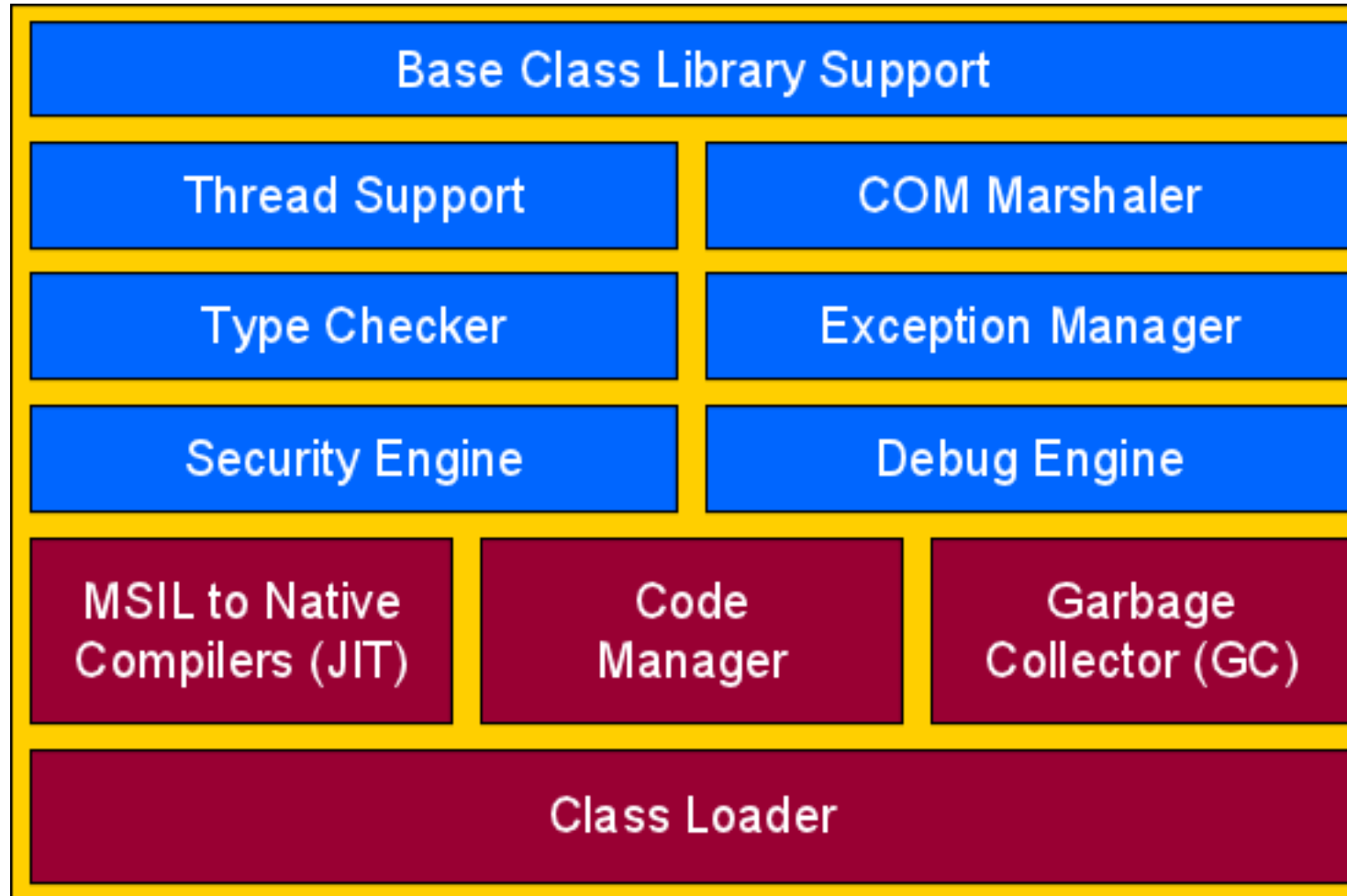
- C# is executed indirectly through an abstract computer architecture called the CLR (Common Language Runtime).
- C# programs are compiled to an IL, which is also called MSIL, CIL (Common Intermediate Language) or bytecode.



CLR and JIT compiling

- The CLR transforms the CIL to assembly instructions for a particular hardware architecture.
 - This is termed Jit'ing or **Just-in-time** compiling.
 - Some initial performance cost, but the jitted code is cached for further execution.
 - The CLR can target the specific architecture in which the code is executing, so some performance gains are possible.
- CLR is an implementation by Microsoft of the CLI (Common Language Infrastructure) which is an open specification, whereas CLR is a platform specific implementation.

Architecture of CLR



Common Language Infrastructure (CLI)

- CLI allows for cross-language development.
- Four components:
 - Common Type System (CTS).
 - Meta-data in a language agnostic fashion.
 - Common Language Specification - behaviors that all languages need to follow.
 - A Virtual Execution System (VES).